

Sure! Below are **all 150 Python questions** followed by their **answers**, categorized and structured for clarity. This will help you rebuild your **logical ability in Python** through practice and understanding.

✓ **PART 1: Basics & Traditional Programs (Q1–Q50)**

Q1. Fibonacci series up to N terms

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=' ')
        a, b = b, a + b
```

Q2. Check Prime

```
def is_prime(n):
    if n <= 1: return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0: return False
    return True
```

Q3. Prime numbers in range

```
for i in range(2, 101):
    if is_prime(i):
        print(i, end=' ')
```

Q4. Factorial

```
def factorial(n):
    result = 1
    for i in range(2, n+1):
        result *= i
    return result
```

Q5. Armstrong number

```
def is_armstrong(n):
    total = sum(int(d)**len(str(n)) for d in str(n))
    return total == n
```

Q6. Palindrome number

```
def is_palindrome(n):
    return str(n) == str(n)[::-1]
```

Q7. Reverse number

```
def reverse_number(n):
    return int(str(n)[::-1])
```

Q8. Sum of digits

```
def sum_digits(n):
```

```
return sum(int(d) for d in str(n))
```

Q9. Count digits

```
def count_digits(n):  
    return len(str(n))
```

Q10. Swap numbers (no temp)

```
a, b = 5, 10  
a, b = b, a
```

Q11. Strong number

```
def is_strong(n):  
    return n == sum(factorial(int(d)) for d in str(n))
```

Q12. Perfect number

```
def is_perfect(n):  
    return sum(i for i in range(1, n) if n % i == 0) == n
```

Q13. First N even numbers

```
def even_numbers(n):  
    return [i*2 for i in range(n)]
```

Q14. First N odd numbers

```
def odd_numbers(n):  
    return [i*2 + 1 for i in range(n)]
```

Q15. LCM

```
def lcm(x, y):  
    from math import gcd  
    return x * y // gcd(x, y)
```

Q16. GCD

```
def gcd(x, y):  
    while y:  
        x, y = y, x % y  
    return x
```

Q17. Decimal to binary

```
bin(10)[2:]
```

Q18. Binary to decimal

```
int('1010', 2)
```

Q19. Multiplication table

```
def table(n):  
    for i in range(1, 11):  
        print(n, '*', i, '=', n*i)
```

Q20. Largest of 3

```
def largest(a, b, c):  
    return max(a, b, c)
```

Q21. Sum of N natural numbers

```
def sum_natural(n):  
    return n * (n + 1) // 2
```

Q22. Sum of N even numbers

```
sum([i*2 for i in range(n)])
```

Q23. Sum of N odd numbers

```
sum([i*2 + 1 for i in range(n)])
```

Q24. AP series

```
def ap(a, d, n):  
    return [a + i*d for i in range(n)]
```

Q25. GP series

```
def gp(a, r, n):  
    return [a * r**i for i in range(n)]
```

Q26. Leap year

```
def is_leap(year):  
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
```

Q27. Count vowels

```
def count_vowels(s):  
    return sum(1 for c in s.lower() if c in 'aeiou')
```

Q28. Factors of a number

```
def factors(n):  
    return [i for i in range(1, n+1) if n % i == 0]
```

Q29. Automorphic number

```
def is_automorphic(n):  
    return str(n*n).endswith(str(n))
```

Q30. Number to words

```
def number_to_words(n):
    mapping = ['Zero', 'One', 'Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine']
    return ' '.join(mapping[int(d)] for d in str(n))
```

Q31. Frequency of digits

```
from collections import Counter
Counter(str(122334))
```

Q32. Power using loop

```
def power(a, b):
    result = 1
    for _ in range(b):
        result *= a
    return result
```

Q33. Reverse string using loop

```
def reverse_string(s):
    rev = ""
    for ch in s:
        rev = ch + rev
    return rev
```

Q34. Reverse list manually

```
def reverse_list(lst):
    return lst[::-1]
```

Q35. Sort list without sort()

```
def manual_sort(lst):
    for i in range(len(lst)):
        for j in range(i+1, len(lst)):
            if lst[i] > lst[j]:
                lst[i], lst[j] = lst[j], lst[i]
    return lst
```

Q36. Second largest number

```
def second_largest(lst):
    unique = list(set(lst))
    unique.sort()
    return unique[-2]
```

Q37. Count even and odd

```
def count_even_odd(lst):
    even = sum(1 for i in lst if i % 2 == 0)
    return even, len(lst)-even
```

Q38. Common elements

```
def common_elements(l1, l2):
    return list(set(l1) & set(l2))
```

Q39. Anagram check

```
def is_anagram(s1, s2):  
    return sorted(s1) == sorted(s2)
```

Q40. Palindrome string

```
def is_palindrome_str(s):  
    return s == s[::-1]
```

Q41. Frequency of characters

```
from collections import Counter  
Counter('hello')
```

Q42. Replace vowels

```
def replace_vowels(s):  
    return ''.join('*' if c in 'aeiouAEIOU' else c for c in s)
```

Q43. ASCII of characters

```
[ord(c) for c in 'hello']
```

Q44. Right triangle pattern

```
for i in range(1, 6):  
    print('*' * i)
```

Q45. Inverted triangle

```
for i in range(5, 0, -1):  
    print('*' * i)
```

Q46. Pascal's triangle

```
def pascal(n):  
    for i in range(n):  
        num = 1  
        for j in range(i + 1):  
            print(num, end=" ")  
            num = num * (i - j) // (j + 1)  
        print()
```

Q47. Diamond pattern

```
for i in range(5):  
    print(' ' * (4-i) + '*' * (2*i+1))  
for i in range(3, -1, -1):  
    print(' ' * (4-i) + '*' * (2*i+1))
```

Q48. Floyd's triangle

```
n = 1
```

```
for i in range(1, 6):
    for j in range(i):
        print(n, end=' ')
        n += 1
    print()
```

Q49. Primes in 2D matrix

```
matrix = [[2, 4], [5, 9]]
for row in matrix:
    print([n for n in row if is_prime(n)])
```

Q50. Merge two sorted lists

```
def merge_sorted(l1, l2):
    return sorted(l1 + l2)
```

Great! Let's continue with:

PART 2: Strings, Patterns, and ASCII Logic (Q51–Q100)

Q51. Reverse a string

```
def reverse_string(s):
    return s[::-1]
```

Q52. Check if a string is palindrome

```
def is_palindrome(s):
    return s == s[::-1]
```

Q53. Count vowels and consonants

```
def count_vowels_consonants(s):
    vowels = 'aeiouAEIOU'

    v = sum(1 for c in s if c in vowels)

    c = sum(1 for c in s if c.isalpha() and c not in vowels)

    return v, c
```

Q54. Count characters, words, spaces

```
def analyze_string(s):  
    return len(s), len(s.split()), s.count(' ')
```

Q55. Frequency of characters

```
from collections import Counter  
  
def char_frequency(s):  
    return Counter(s)
```

Q56. Remove vowels from string

```
def remove_vowels(s):  
    return ''.join(c for c in s if c.lower() not in 'aeiou')
```

Q57. Replace spaces with hyphens

```
def replace_spaces(s):  
    return s.replace(' ', '-')
```

Q58. Remove special characters

```
import re  
  
def remove_special(s):  
    return re.sub('[^A-Za-z0-9 ]+', '', s)
```

Q59. Capitalize first letter of each word

```
def capitalize_words(s):  
    return s.title()
```

Q60. Swap case

```
def swap_case(s):  
    return s.swapcase()
```

Q61. Remove duplicate characters

```
def remove_duplicates(s):  
    return ''.join(sorted(set(s), key=s.index))
```

Q62. Check anagrams

```
def are_anagrams(s1, s2):  
    return sorted(s1) == sorted(s2)
```

Q63. Longest word in sentence

```
def longest_word(s):  
    return max(s.split(), key=len)
```

Q64. Sort words in string

```
def sort_words(s):  
    return ' '.join(sorted(s.split()))
```

Q65. Reverse each word

```
def reverse_words(s):  
    return ' '.join(word[::-1] for word in s.split())
```

Q66. Title case conversion

```
def to_title_case(s):  
    return s.title()
```

Q67. Contains only digits

```
def is_digit_only(s):  
    return s.isdigit()
```

Q68. First non-repeating character

```
from collections import Counter
```

```
def first_non_repeat(s):
```

```
    count = Counter(s)
```

```
    for c in s:
```

```
        if count[c] == 1:
```

```
            return c
```

Q69. Count uppercase and lowercase

```
def count_cases(s):
```

```
    return sum(1 for c in s if c.isupper()), sum(1 for c in s if c.islower())
```

Q70. ASCII of characters

```
def ascii_values(s):
```

```
    return [ord(c) for c in s]
```

Q71. Caesar cipher

```
def caesar_encrypt(s, shift):
```

```
    return ''.join(chr((ord(c)-65+shift)%26 + 65) if c.isupper() else c for c in s)
```

Q72. Frequency of words

```
from collections import Counter
```

```
def word_count(s):
```

```
    return Counter(s.split())
```

Q73. Pangram check

```
def is_pangram(s):
```

```
    return set('abcdefghijklmnopqrstuvwxyz').issubset(set(s.lower()))
```

Q74. Star pattern - triangle

for i in range(1, 6):

```
    print('*' * i)
```

Q75. Inverted triangle

for i in range(5, 0, -1):

```
    print('*' * i)
```

Q76. Hollow rectangle

def hollow_rectangle(rows, cols):

```
    for i in range(rows):
```

```
        for j in range(cols):
```

```
            if i in [0, rows-1] or j in [0, cols-1]:
```

```
                print('*', end="")
```

```
            else:
```

```
                print(' ', end="")
```

```
        print()
```

Q77. Diamond star

n = 5

for i in range(n):

```
    print(" "*(n-i-1) + "*"*(2*i+1))
```

for i in range(n-2, -1, -1):

```
    print(" "*(n-i-1) + "*"*(2*i+1))
```

Q78. Number pyramid

n = 5

for i in range(1, n+1):

```
    print(' '.join(str(j) for j in range(1, i+1)))
```

Q79. Alphabet triangle

```
for i in range(1, 6):  
    print(''.join(chr(65 + j) for j in range(i)))
```

Q80. Staircase pattern

```
for i in range(1, 6):  
    print(' '*(5-i) + '#' * i)
```

Q81. Alternating 1 and 0

```
for i in range(1, 6):  
    print(''.join('1' if (i+j)%2==0 else '0' for j in range(i)))
```

Q82. Hollow triangle

```
n = 5  
  
for i in range(1, n+1):  
    for j in range(1, i+1):  
        if j == 1 or j == i or i == n:  
            print("*", end="")  
        else:  
            print(" ", end="")  
    print()
```

Q83. Floyd's triangle

```
n = 1  
  
for i in range(1, 6):  
    for j in range(i):  
        print(n, end=' ')  
        n += 1  
    print()
```

Q84. Alphabet pyramid

```
for i in range(1, 6):  
    print(''.join(chr(65 + j) for j in range(i)))
```

Q85. Chessboard pattern

```
for i in range(8):  
    print(''.join('# ' if (i+j)%2==0 else ' ' for j in range(8)))
```

Q86. Continuous numbers triangle

```
n = 1  
  
for i in range(1, 6):  
    for j in range(i):  
        print(n, end=' ')  
        n += 1  
    print()
```

Q87. Mirror triangle

```
for i in range(1, 6):  
    print(' ' * (5 - i) + '*' * i)
```

Q88. Hourglass pattern

```
for i in range(5, 0, -1):  
    print(' ' * (5-i) + '*' * i)  
  
for i in range(2, 6):  
    print(' ' * (5-i) + '*' * i)
```

Q89. Zig-zag pattern

```
for i in range(3):  
    for j in range(15):  
        if ((i + j) % 4 == 0) or (i == 2 and j % 4 == 0):
```

```

        print("", end="")

    else:

        print(" ", end="")

    print()

```

Q90. Multiplication triangle

```

for i in range(1, 6):

    for j in range(1, i+1):

        print(f'{i*j:2}', end=' ')

    print()

```

Q91. Manual string reverse

```

def manual_reverse(s):

    rev = ""

    for ch in s:

        rev = ch + rev

    return rev

```

Q92. Extract numbers

```

import re

re.findall(r'\d+', 'abc123xyz45')

```

Q93. Compare strings without ==

```

def are_equal(s1, s2):

    return all(a == b for a, b in zip(s1, s2)) and len(s1) == len(s2)

```

Q94. Validate password strength

```

import re

def validate_password(p):

    return bool(re.match(r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$]).{8,}$', p))

```

Q95. Remove extra spaces

```
def clean_spaces(s):  
    return ' '.join(s.split())
```

Q96. Print string vertically

```
def vertical_string(s):  
    for ch in s:  
        print(ch)
```

Q97. Print each word

```
def word_per_line(s):  
    for word in s.split():  
        print(word)
```

Q98. String to list of words

```
def string_to_list(s):  
    return s.split()
```

Q99. Format string

```
def greet(name):  
    return "Hello, {}".format(name)
```

Q100. Replace substring

```
def replace_sub(s, old, new):  
    return s.replace(old, new)
```

Awesome! Here's the final section:

PART 3: Lists, Tuples, Dictionaries, Sets (Q101–Q125)

Q101. Sum of list elements

```
def sum_list(lst):  
    return sum(lst)
```

Q102. Find max and min

```
def max_min(lst):  
    return max(lst), min(lst)
```

Q103. Remove duplicates

```
def remove_duplicates(lst):  
    return list(set(lst))
```

Q104. Reverse list manually

```
def reverse_list(lst):  
    return lst[::-1]
```

Q105. Count occurrences

```
from collections import Counter  
  
def count_occurrences(lst):  
    return Counter(lst)
```

Q106. Second largest number

```
def second_largest(lst):  
    return sorted(set(lst))[-2]
```

Q107. Merge two lists

```
def merge_lists(l1, l2):  
    return l1 + l2
```

Q108. Common elements

```
def common_elements(l1, l2):  
    return list(set(l1) & set(l2))
```

Q109. Split list into even and odd

```
def split_even_odd(lst):  
    even = [x for x in lst if x % 2 == 0]  
    odd = [x for x in lst if x % 2 != 0]  
    return even, odd
```

Q110. Elements at even indexes

```
def even_index_elements(lst):  
    return lst[::2]
```

Q111. Sum of even numbers

```
def sum_even(lst):  
    return sum(x for x in lst if x % 2 == 0)
```

Q112. Multiply all numbers

```
def product(lst):  
    result = 1  
    for x in lst:  
        result *= x  
    return result
```

Q113. Average of list

```
def average(lst):  
    return sum(lst) / len(lst)
```

Q114. Remove None values

```
def remove_none(lst):  
    return [x for x in lst if x is not None]
```

Q115. Manual sort

```
def bubble_sort(lst):  
    for i in range(len(lst)):  
        for j in range(len(lst)-1-i):  
            if lst[j] > lst[j+1]:  
                lst[j], lst[j+1] = lst[j+1], lst[j]  
    return lst
```

Q116. List of squares

```
def squares(n):  
    return [i**2 for i in range(1, n+1)]
```

Q117. Tuple unpacking

```
a, b, c = (1, 2, 3)
```

Q118. Tuple contains value

```
def contains(tup, val):  
    return val in tup
```

Q119. List to tuple and back

```
tuple1 = tuple([1, 2, 3])  
list1 = list(tuple1)
```

Q120. Count duplicates in tuple

```
from collections import Counter
```

```
def count_tuple(t):  
    return Counter(t)
```

Q121. Iterate dictionary

```
def print_dict(d):  
    for k, v in d.items():  
        print(k, v)
```

Q122. Word frequency dictionary

```
from collections import Counter
```

```
def word_freq(text):  
    return Counter(text.split())
```

Q123. Merge dictionaries

```
def merge_dicts(d1, d2):  
    return {**d1, **d2}
```

Q124. Key exists in dictionary

```
def key_exists(d, key):  
    return key in d
```

Q125. Remove duplicate dictionary values

```
def remove_dup_values(d):  
    seen, new = set(), {}  
  
    for k, v in d.items():  
        if v not in seen:
```

```
seen.add(v)

new[k] = v

return new
```

✓ PART 4: Functions, Recursion, Math & Puzzles (Q126–Q150)

Q126. Prime check with function

```
def is_prime(n):
    if n <= 1: return False

    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False

    return True
```

Q127. Factorial using function

```
def factorial(n):
    return 1 if n==0 else n * factorial(n-1)
```

Q128. Fibonacci recursion

```
def fib(n):
    if n <= 1: return n

    return fib(n-1) + fib(n-2)
```

Q129. Factorial recursion

```
def fact(n):
    return 1 if n == 0 else n * fact(n - 1)
```

Q130. GCD of list

```
from math import gcd

from functools import reduce

def list_gcd(lst):

    return reduce(gcd, lst)
```

Q131. Palindrome check function

```
def is_palindrome(s):

    return s == s[::-1]
```

Q132. Lambda for square

```
square = lambda x: x**2
```

Q133. Sort list of tuples by 2nd element

```
def sort_tuples(lst):

    return sorted(lst, key=lambda x: x[1])
```

Q134. Selection sort

```
def selection_sort(lst):

    for i in range(len(lst)):

        min_idx = i

        for j in range(i+1, len(lst)):

            if lst[j] < lst[min_idx]:

                min_idx = j

        lst[i], lst[min_idx] = lst[min_idx], lst[i]

    return lst
```

Q135. Bubble sort

```
def bubble_sort(lst):

    for i in range(len(lst)):

        for j in range(len(lst)-1-i):
```

```
        if lst[j] > lst[j+1]:
            lst[j], lst[j+1] = lst[j+1], lst[j]

    return lst
```

Q136. Insertion sort

```
def insertion_sort(lst):
    for i in range(1, len(lst)):
        key = lst[i]
        j = i - 1
        while j >= 0 and key < lst[j]:
            lst[j + 1] = lst[j]
            j -= 1
        lst[j + 1] = key
    return lst
```

Q137. Linear search

```
def linear_search(lst, target):
    for i, val in enumerate(lst):
        if val == target:
            return i
    return -1
```

Q138. Binary search

```
def binary_search(lst, target):
    low, high = 0, len(lst)-1
    while low <= high:
        mid = (low + high) // 2
        if lst[mid] == target:
            return mid
        elif lst[mid] < target:
```

```
        low = mid + 1

    else:

        high = mid - 1

    return -1
```

Q139. Median of list

```
def find_median(lst):

    lst.sort()

    n = len(lst)

    return (lst[n//2] + lst[(n-1)//2]) / 2 if n%2 == 0 else lst[n//2]
```

Q140. Mode of list

```
from collections import Counter

def mode(lst):

    return Counter(lst).most_common(1)[0][0]
```

PART 5: Logical Problems (Q141–Q150)

Q141. Check if two lists are permutations

```
def are_permutations(l1, l2):

    return sorted(l1) == sorted(l2)
```

Q142. All pairs with given sum

```
def find_pairs(lst, k):

    pairs = []

    seen = set()

    for num in lst:

        if k - num in seen:
```

```
        pairs.append((k - num, num))

    seen.add(num)

    return pairs
```

Q143. Missing number in 1 to N

```
def missing_number(lst, n):

    return n*(n+1)//2 - sum(lst)
```

Q144. Find duplicate number

```
def find_duplicate(lst):

    seen = set()

    for num in lst:

        if num in seen:

            return num

    seen.add(num)
```

Q145. Rotate list by K

```
def rotate_list(lst, k):

    k %= len(lst)

    return lst[-k:] + lst[:-k]
```

Q146. Detect cycle in list

```
def has_cycle(lst):

    slow = fast = 0

    while fast < len(lst) and fast+1 < len(lst):

        slow += 1

        fast += 2

        if lst[slow % len(lst)] == lst[fast % len(lst)]:

            return True

    return False
```

Q147. Balanced brackets

```
def is_balanced(s):  
    stack = []  
  
    mapping = {'(': '(', ')': ')', '[': '[', ']': ']' }  
  
    for c in s:  
        if c in '({[':  
            stack.append(c)  
  
        elif stack and stack[-1] == mapping[c]:  
            stack.pop()  
  
        else:  
            return False  
  
    return not stack
```

Q148. All subsets of list

```
from itertools import chain, combinations  
  
def all_subsets(lst):  
    return list(chain.from_iterable(combinations(lst, r) for r in range(len(lst)+1)))
```

Q149. Prime factors of number

```
def prime_factors(n):  
    i = 2  
  
    factors = []  
  
    while i <= n:  
        if n % i == 0:  
            factors.append(i)  
  
            n //= i  
  
        else:  
            i += 1  
  
    return factors
```

Q150. Max profit stock prices

```
def max_profit(prices):  
    min_price = float('inf')  
  
    profit = 0  
  
    for price in prices:  
        min_price = min(min_price, price)  
  
        profit = max(profit, price - min_price)  
  
    return profit
```